

HEIG-VD

19.03.2026



Florian Duruz et Raphaël Perret

Dataflow

Dataflow est un paradigme de programmation où les données traversent le programme de manière asynchrone. Le programme peut alors être représenté par un graphe orienté traversé par des données. Chaque sommet ou nœud s'active dès que ses prérequis sont remplis.

Au niveau logiciel, le paradigme DataFlow conserve les mêmes principes fondamentaux, appliqués à l'organisation du code :

- Un programme est représenté comme un graphe orienté de nœuds de traitement.
- Les données circulent sur les arcs reliant les nœuds sous forme de messages (tokens).
- Un nœud s'active dès que toutes ses entrées sont disponibles, pas avant.
- Chaque nœud est une fonction pure : mêmes entrées produisent toujours les mêmes sorties, sans effet de bord.

Comparaison avec l'architecture impérative

La distinction fondamentale entre les deux paradigmes porte sur qui décide de l'ordre d'exécution.

Architecture impérative	Paradigme DataFlow
Le développeur dicte l'ordre d'exécution	Le flux des données dicte quand chaque nœud s'exécute
Les données attendent passivement en mémoire	Les données sont actives : elles déclenchent l'exécution
L'état est partagé et mutable	Chaque nœud produit une sortie immuable
Le parallélisme est une responsabilité manuelle	Le parallélisme est une conséquence naturelle de la pureté
Ajouter une fonctionnalité modifie le code existant	Ajouter une fonctionnalité = ajouter un nœud et un lien

La règle de déclenchement

La règle centrale du DataFlow, dite "règle de déclenchement", stipule qu'un nœud s'active dès que toutes ses entrées sont satisfaites. Cette règle implique deux propriétés essentielles :

- **Pureté.** La pureté fonctionnelle : un nœud ne peut pas modifier l'état d'un autre nœud. Il reçoit des données, calcule, et émet un résultat. C'est tout.
- **Parallélisme.** Le parallélisme implicite : deux nœuds dont les entrées sont indépendantes peuvent s'exécuter simultanément sans coordination explicite.

Application au netcode : State Sync vs DataFlow

Dans le contexte multijoueur, deux paradigmes s'affrontent pour la synchronisation réseau :

State Synchronization (approche classique)	DataFlow + Server Authority (approche projet)
Synchronise un état résultant	Propage une intention ayant causé le changement
Le client reçoit : "la position vaut (4,2)"	Le serveur reçoit : "le joueur a exécuté MoveRight"
Répond à : quelle est la valeur maintenant ?	Répond à : pourquoi la valeur a changé ?
Exemples : Unity NetworkVariable, Unreal RepNotify	Exemples : TPL Dataflow
Variables envoyées séparément : risque d'incohérence	Snapshot atomique : état toujours cohérent

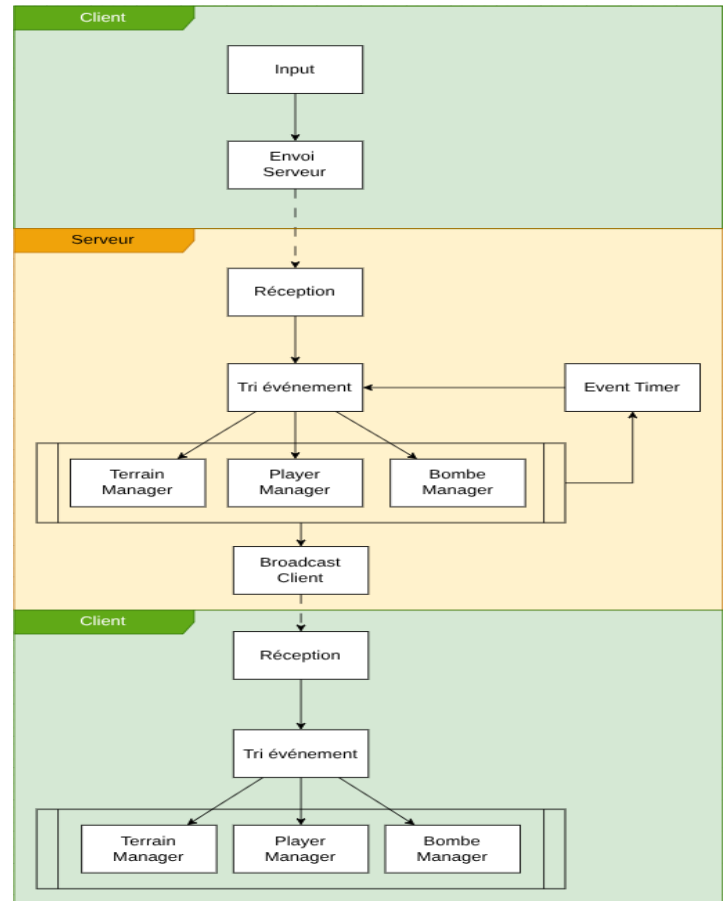
Point critique : le projet adopte un modèle Server Authority. Le client envoie une intention au serveur. Le pipeline DataFlow du serveur la valide, la traite, et diffuse un WorldSnapshot immuable à tous les clients. Les clients appliquent le snapshot sans recalcul ; l'état est cohérent par construction.

Motivations

Nous sommes convaincus qu'il est possible d'obtenir une architecture propre et fonctionnelle avec Dataflow pour un jeu multijoueur. Un programme Dataflow pouvant être représenté sous forme de graphe, nous avons représenté notre problème sous forme de diagramme de flux. Ci-contre le diagramme dans son entièreté montre l'architecture dans sa globalité, du moins pour la gestion d'une partie en cours. Les rectangles représentent des blocks Dataflow, les flèches pleines des données passant d'un bloc à l'autre dans un même processus. Les flèches en traitiez représentent une communication sur le réseau.

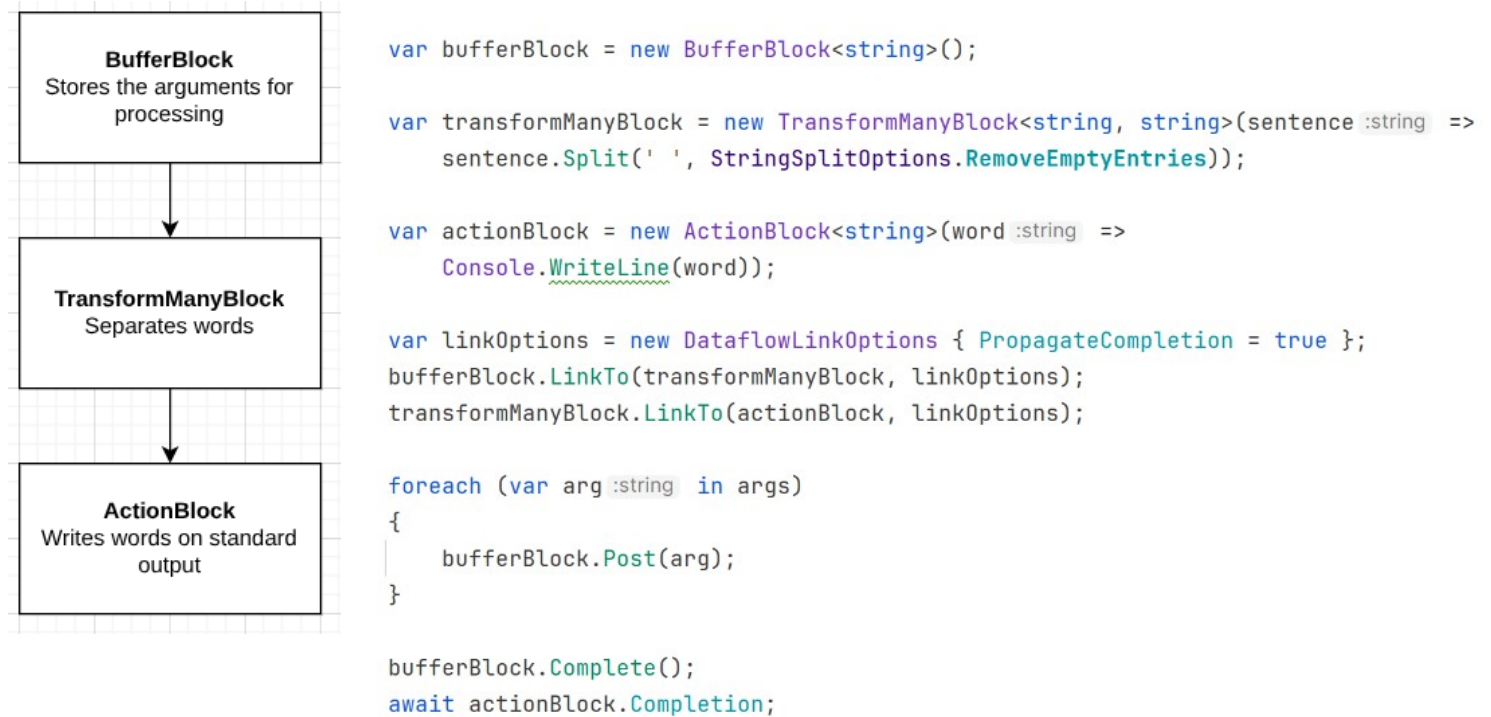
Lorsque le joueur appuie sur une touche, l'évènement est envoyé au serveur sans mettre à jour l'état du jeu chez le client. On espère avoir une latence suffisamment faible pour que cela ne se remarque pas, mais le cas échéant, nous devrions temporairement modifier l'état local pour donner un sentiment de réactivité au joueur. Le serveur reçoit un message du client et le traite comme un évènement et le distribue au bon manager selon son type. Le manager concerné met à jour son état et éventuellement ceux de systèmes connexes, puis envoie le nouvel état du jeu à tous les clients. Il calcule également le prochain évènement qui se produira s'il n'y a pas d'intervention d'un joueur et l'ajoute après le temps calculé aux évènements à traiter. Ainsi dans le cas d'une bombe dont l'explosion est imminente, un évènement de type "bombe explose" est ajouté après un temps équivalent au temps restant avant son explosion théorique. Le serveur fonctionne alors purement sur le principe du Dataflow : les données, appelées évènements dans ce cas déclenchent les blocs du programme de manière asynchrone.

Les clients reçoivent le nouvel état du jeu et mettent à jour les managers concernés de la même manière que le serveur. Il y a un potentiel pour réutiliser les mêmes composants de logique de jeu sur le serveur et dans les clients. Ces derniers devront enfreindre le paradigme pour la gestion de l'affichage. En effet, un jeu moderne devrait atteindre une fréquence de rafraîchissement d'au moins soixante images par secondes et on imagine mal le serveur déclencher ces rafraîchissements. Il y aura donc une partie procédurale pour l'affichage et l'interpolation entre les états envoyés par le serveur. On va pouvoir encapsuler les mécanismes d'écriture et de lecture des sockets TCP dans des blocs producteurs et consommateurs pour abstraire la logique réseau dans la philosophie Dataflow.



Langage

Nous avons choisi d'utiliser C# pour deux raisons principales : les bibliothèques et frameworks pour le jeu vidéo et la bibliothèque TPL de Microsoft incluant des blocs Dataflow. Pour comprendre comment un programme Dataflow est structuré en C# avec la bibliothèque TPL, nous avons implémenté un programme "echoIn" simple : le programme affiche chaque mot de chaque argument sur la sortie standard. On aura besoin d'un bloc de départ dans lequel écrire les arguments, d'un bloc pour les séparer en mots puis enfin d'un bloc d'action pour les afficher. Il est résumé sur ce diagramme que nous pouvons directement comparer avec le code.



Dans le code, on remarque une première partie où on ne fait que déclarer des blocs et les opérations qu'ils effectuent. Dans un deuxième temps, on lie les blocs ensemble avec la méthode `LinkTo`, puis enfin on insère des données dans le premier bloc de la chaîne et on attend que le dernier bloc d'action ait fini. On peut ici attendre la complétion de la tâche car on indique explicitement qu'il n'y aura pas de nouvelles données avec la méthode `Complete` et on a demandé la propagation de la complétion avec l'option de lien `PropagateCompletion = true`. Ainsi, lorsque le bloc buffer reçoit l'information que sa tâche est finie, il informe le/les blocs suivants qu'ils ne recevront plus de nouvelles données, en appelant leurs méthode `Complete`.

Étudions maintenant les interfaces qui définissent les blocs Dataflows. `IDataFlowBlock` définit la propriété `Completion`, qui retourne un objet `Task` qui représente la tâche asynchrone du bloc, les méthodes `Complete`, qui indique au bloc qu'il ne recevra plus rien et `Fault` qui termine la tâche asynchrone mais avec une erreur. Deux interfaces étendent `IDataFlowBlock`, `ISourceBlock` et `ITargetBlock`. `ISourceBlock` représente une source de données pour le flux Dataflow, dont les messages devront être consommés par un objet implémentant l'interface `ITargetBlock`. C'est donc `ISourceBlock` qui offre la méthode `LinkTo`, et l'interface `ITargetBlock` qui offre la méthode `Post`. Logiquement, tout bloc intermédiaire implémente les deux interfaces. C'est le cas de `BufferBlock` et `TransformManyBlock` dans l'exemple précédent, mais pas de `ActionBlock`, qui est purement un consommateur.

Cahier des charges

Serveur

Le serveur écoute les requêtes des clients et traite la logique du jeu puis renvoi les modifications aux clients, ce processus se fera via un bloc dataflow dédié, pour la réception des requêtes et pour le broadcast des mises à jour aux clients.

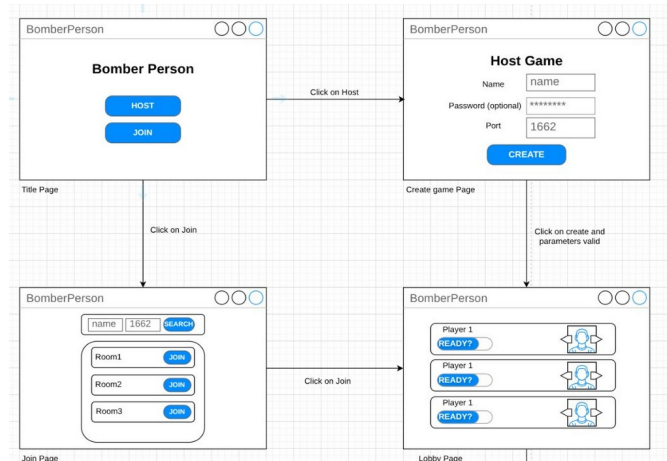
Le serveur est responsable de garder en tout temps un état du jeu valide et à jour. Chaque modification de cet état provoque l'envoi de l'état modifié à tous les clients. Une modification peut être déclenchée par la réception d'un message d'un client ou lors de la résolution d'un évènement (explosion de bombe, diminution du terrain de jeu...)

Client

Le client affiche des pages de menus permettant de découvrir des parties sur le réseau local et d'en créer. Une fois une partie découverte, il permet de la rejoindre, de choisir un avatar et de se déclarer prêt à commencer. Une fois que la partie a démarrée, le client communique avec le serveur et affiche le jeu. Détaillons maintenant chaque aspect du client.

Menus

- Une première page d'accueil permet de choisir entre héberger une partie ou en rejoindre une.
- La page de découverte montre toutes les parties trouvées sur le réseau local et permet d'en rejoindre une
- La page de création permet de créer une partie et de la paramétrer
- La page d'attente propose aux joueurs de choisir un avatar et de se déclarer prêt



Ce diagramme est une représentation des menus et de la navigation entre eux d'une manière très synthétique et ne représentant pas l'aspect visuel final.

En Jeu

Le client reçoit des états de la partie du serveur et mets à jour le sien pour s'y conformer. Lorsque le joueur effectue des actions, celles-ci sont envoyées au serveur. Le client n'est pas responsable de calculer un nouvel état valide du jeu ou les évènements à résoudre.

Pour garantir un affichage assez rapide, le client interpole entre les états reçus. Il ne vérifie pas la nécessairement validité de ces états interpolés. Il n'est pas impossible de créer des situations injustes où le client interpole trop à partir d'un état connu et la situation est visuellement invalide, mais la plupart des jeux multijoueur font certains compromis à cet égard.

Scope

- **Déplacement des joueurs** : deux joueurs se déplacent sur la grille, en local et via le réseau.
- **Pose et explosion de bombes** : un joueur pose une bombe, elle explose après un timer et détruit les blocs adjacents.
- **Destruction du terrain** : les blocs destructibles sont éliminés par les explosions, la grille est mise à jour.
- **Multijoueur en réseau LAN**: deux instances du jeu communiquent via TCP, architecture Server Authority.

Contraintes non fonctionnelles

Si seul le serveur calcule de nouveaux états de jeu selon les actions des joueurs, nous devons comprendre les différentes latences impliquées. Nous nous limitons aux réseaux LAN, qui offrent une latence autour de 1ms, avec des équipements modernes. Les joueurs sont limités par la fréquence de leurs claviers, qui peut atteindre 8KHz. Donc on aurait le temps de provoquer 10 évènements sur un seul client avant que le serveur en reçoive une seule. Cela est serait évidemment un problème si un humain était capable de cliquer aussi rapidement, mais ce n'est pas le cas : une personne moyenne ne dépasse pas les 50 cliques par secondes sur une seule touche. Cela nous laisse une fenêtre de 20ms pour traiter la requête avant que la suivante arrive, ce qui semble très réaliste. Le serveur doit donc répondre à une requête en moins de 10ms pour respecter cette contrainte.

Nous n'avons aucune exigence particulière en matière de sécurité ; on admet que les joueurs utilisent notre client sans le modifier et qu'ils n'interfèrent pas d'une autre manière sur le serveur.